

Flex Box

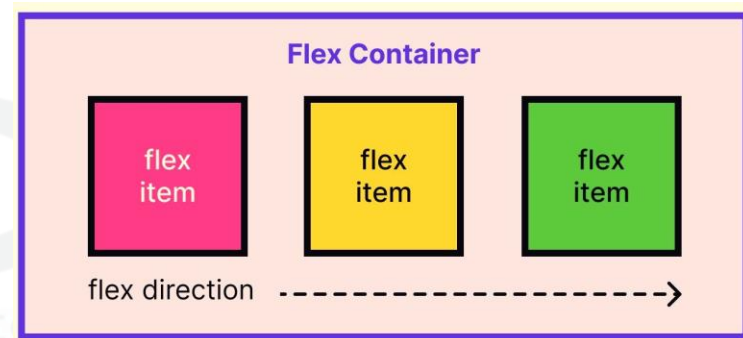
Flexbox (short for *Flexible Box*) is a **CSS layout technique** that helps you **arrange items in a row or column** and control how they **grow, shrink, or wrap** — even when the screen size changes.

Set the container to use Flex:

```
.container {  
  display: flex;  
}
```

Put items inside the container:

```
<div class="container">  
  <div class="item">A</div>  
  <div class="item">B</div>  
  <div class="item">C</div>  
</div>
```



Flexbox Properties



On the Container (.container)

Property	Meaning
display: flex;	Makes the container a flexbox.
flex-direction	Direction of items: row (default), column, row-reverse.
justify-content	Horizontal alignment: flex-start, center, space-between.
align-items	Vertical alignment: stretch, center, flex-start, etc.
flex-wrap	Allows items to wrap to next line: wrap or nowrap.

Flexbox Properties



On the Items (.item)

Property	Meaning
flex	How much space an item should take compared to others.
align-self	Override vertical alignment for just one item.
order	Change the position of an item without changing the HTML order.

Example:

```
<style>
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100px;
  background: lightblue;
}
.item {
  background: orange;
  padding: 10px;
  margin: 5px;
}
</style>

<div class="container">
  <div class="item">Item 1</div>
  <div class="item">Item 2</div>
</div>
```

-webkit



WebKit is a **browser engine** – it's like the brain behind how your webpage is displayed in a browser.

Imagine your browser (like Chrome or Safari) is a movie theatre, and the **HTML/CSS/JS** is your movie script. **WebKit** is the **director** who reads the script and shows it on the screen.

Why do we see -webkit- in CSS?

Because **different browsers** understand CSS in slightly different ways, developers use **browser-specific prefixes** to make sure it works everywhere. **Common Vendor Prefixes:**

Prefix	For Browser
-webkit-	Chrome, Safari
-moz-	Firefox
-ms-	Internet Explorer
-o-	Opera

-webkit

```
.box {  
  -webkit-border-radius: 10px; /* for WebKit browsers */  
  border-radius: 10px;        /* standard CSS */  
}
```

Why is it still used?

Even though **modern browsers** understand most standard CSS, we **still use -webkit-** sometimes:

- To support **older browsers**.
- For **experimental features** not fully supported yet.

Example:

```
<style>
.box {
  width: 150px;
  height: 150px;
  background: orange;

  /* WebKit-specific */
  -webkit-border-radius: 50%;

  /* Standard */
  border-radius: 50%;
}
</style>

<div class="box"></div>
```

Pseudo-Class & Element



A **pseudo-class (:)** is used to **style an element based on its state or position** — without needing to add a class or ID in HTML.

```
a: hover {  
  color: red;  
}
```

A **pseudo-element (::)** lets you **style a specific part of an element, as if it's a fake (pseudo) element** inside it — even if it doesn't exist in the actual HTML.

```
p::first-letter {  
  font-size: 200%;  
}
```


Pseudo-Class

Pseudo-Class	What it Does	Example
:hover	When the mouse is over the element	button:hover
:active	When the element is being clicked	a:active
:focus	When the element is focused	input:focus
:visited	Already visited links	a:visited
:first-child	The first child of a parent	li:first-child
:last-child	The last child of a parent	li:last-child
:nth-child(n)	The nth child	li:nth-child(3)
:not(selector)	All elements except this one	p:not(.special)
:checked	When a checkbox/radio is selected	input:checked
:disabled	Disabled input fields	input:disabled
:enabled	Enabled input fields	input:enabled
:required	Required input fields	input:required
:valid / :invalid	Valid or invalid form input	input:valid / input:invalid
:empty	Empty elements	div:empty
:target	Element with an ID that is currently targeted (e.g., via anchor)	section:target

Example:

```
<style>
  a:hover {
    color: red;
  }

  input:focus {
    border: 2px solid blue;
  }

  ul li:first-child {
    font-weight: bold;
  }

  input:checked {
    outline: 2px solid green;
  }
</style>
```

```
<a href="#">Hover over this link</a><br><br>
<input type="text" placeholder="Click me"><br><br>
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
<input type="checkbox"> Check me
```

Pseudo-Element

Pseudo-Element	What it targets
::before	Adds content before an element
::after	Adds content after an element
::first-letter	Styles the first letter of text
::first-line	Styles the first line of text
::selection	Styles the part of text the user selects
::marker (new)	Styles list item bullet or number
::placeholder	Styles the placeholder text of input
::backdrop	Styles modal background (e.g. <dialog>)
::cue	Styles WebVTT captions (used in video)

Important Notes

- Pseudo-elements **do not exist in HTML**, but are **styled through CSS**.
- ::before and ::after **require** a content property to work.
- Modern CSS uses :: (double colon), but older browsers allowed : too (:before, :after).

Example:

```
<style>
h2::before {
  content: "👉 ";
}
h2::after {
  content: "✅ ";
}
p::first-letter {
  font-size: 200%;
  color: red;
}
::selection {
  background-color: gold;
  color: black;
}
</style>
```

<h2>Hello World</h2>

<p>This is a paragraph to test pseudo-elements. Try selecting some text.</p>